

SUPCONTENT Documentation Technique

Juin 2025

Sommaire

1. Installation et prérequis
2. Guide de déploiement
3. Justification des choix technologiques
4. Diagrammes UML
5. Schéma de la base de données

1. Installation et prérequis

1.1 Matériel

Vous avez besoin d'un environnement de développement complet comprenant

Docker : 24.X

Docker Compose : 2.X

Git : 2.X

Un éditeur de texte style VSCode, IntelliJ

1.2 Clé d'API

Pour utiliser pleinement SUPCONTENT en local vous aurez besoin d'une Clé d'API , pour cela rendez vous sur :

- <https://www.themoviedb.org/>

afin d'obtenir votre clé d'API TheMovieDB

2. Guide de déploiement

2.1 Via Git

Créer un dossier SUPCONTENT ou vous le voulez et dans un terminal :

- git clone <https://github.com/Miklva/3PROJ.git>

Dupliquez le fichier .env.example en .env et renseignez vos identifiants personnelles, dont la clé d'API TheMovieDB que l'on a crée .

- docker compose up --build

L'application est désormais disponible sur : <http://localhost:3000/>

2.2 Deploiement pour développement

Dans un 1^{er} terminal :

```
cd server  
npm install  
npm start
```

Et dans un second :

```
cd web  
npm install  
npm run dev
```

Pour pouvoir developper sans recharger en permanence les dockers

3. Justification des choix technologiques

3.1 API

Nous avons utilisé TheMovieDB pour ces choix :

- Gratuité
- Richesse des données
- Documentation
- Traduction

3.2 Backend / Frontend

Le backend est développé en JavaScript avec Node.js et le framework Express.js. Ce choix est motivé par :

- Écosystème unifié
- Socket.io

Le client web est construit avec React 18, Vite comme bundler, et TypeScript pour le typage statique:

3.4 Application Mobile

Le client mobile est développé avec React Native, permettant de cibler iOS et Android depuis une base de code unique :

- Partage de compétences
- Cross-platform
- Composants natifs

3.5 Base de données

La base de données relationnelle MySQL 8.0 a été choisie pour les raisons suivantes :

- Modèle relationnel adapté
- Intégrité référentielle

4.Diagrammes UML

4.1 Diagramme de cas d'utilisation

Le diagramme suivant présente les interactions entre les différents acteurs et les fonctionnalités principales de SUPCONTENT.

Acteur	Cas d'utilisation
Visiteur (non connecté)	Consulter les fiches films/séries Rechercher une œuvre Consulter les profils publics Consulter les listes publiques
Utilisateur connecté	Tout ce que fait le Visiteur, plus : S'inscrire / Se connecter (email ou OAuth2) Gérer sa bibliothèque (statuts) Créer des listes personnalisées Rédiger et noter des avis Suivre / Ne plus suivre un utilisateur Interagir avec les avis (like, commentaire) Consulter son fil d'actualité Recevoir des notifications Envoyer des messages privés Exporter ses données (RGPD)
Administrateur	Tout ce que fait l'Utilisateur, plus : Modérer les contenus signalés Bannir des utilisateurs Mettre en avant des avis (Coup de cœur)

4.2 Diagramme de séquence

Ce diagramme décrit le flux complet lors d'une recherche d'œuvre par un utilisateur, illustrant le rôle de cache du serveur SUPCONTENT vis-à-vis de l'API TMDB.

Étape	Acteur	→	Destinataire	Message
1	Client	→	Backend SUPCONTENT	GET /api/media/search/movie?query=Inception
2	Backend	→	Cache MySQL	SELECT * WHERE title LIKE '%Inception%'
3	MySQL	→	Backend	Cache vide ou expiré (>7 jours)
4	Backend	→	API TMDB	GET /search/movie?query=Inception&api_key=...&language=fr-FR
5	API TMDB	→	Backend	JSON : liste de films correspondants
6	Backend	→	MySQL	INSERT INTO media_cache (résultats)
7	Backend	→	Client	JSON : résultats enrichis (données TMDB + notes SUPCONTENT)
8	Client	→	Utilisateur	Affichage de la liste de résultats

5. Schéma de la base de données

La base de données MySQL supcontent est composée de 9 tables. Voici leur description et leurs relations :

Table	Rôle principal	Clés étrangères notables
users	Stocke tous les utilisateurs	-
subscriptions	Système de follow	follower_id → users, following_id → users
reviews	Avis et notes des utilisateurs sur un film/serie	user_id → users
review_likes	J' aime sur une critique	review_id → reviews, user_id → users
review_comments	Commentaires sous une critique	review_id → reviews, user_id → users
reports	Signalements de contenus inappropriés	review_id → reviews, reporter_id → users
lists	Listes personnalisées ou par défaut	user_id → users
list_items	Œuvres ajoutées à une liste	list_id → lists
media_cache	Cache local des données TMDB	-
messages	Messages privés entre utilisateurs	sender_id → users, receiver_id → users

5.1 Relations entre tables

Voici un résumé textuel des relations entre les entités principales :

- Un utilisateur peut rédiger plusieurs avis (1 par œuvre maximum).
- Un avis peut recevoir plusieurs likes et plusieurs commentaires.
- Un utilisateur peut suivre plusieurs autres utilisateurs (many-to-many via subscriptions).
- Un utilisateur possède plusieurs listes (dont 4 listes par défaut créées à l'inscription).
- Une liste contient plusieurs list_items (œuvres issues de TMDB).
- Un utilisateur peut signaler un avis (1 signalement par avis par utilisateur).
- Un utilisateur peut envoyer des messages à d'autres utilisateurs.
- Le cache media est indépendant des utilisateurs et est partagé globalement.